Série TD03 : Liste chaînée *(Corrigé)* 2024

Exercice N°1

Algorithmique

1. **TYPE** noeud = **structure**
   r : réel
   suiv : ^noeud
   Fin structure

2. **Fonction** creernoeud( ) :^noeud
   VAR n :^noeud
   DÉBUT
   n ⟵ allouer(noeud)
   Écrire("Donner une valeur réelle") Lire(n⟶r)
   Renvoyer n
   FIN

3. **Procédure** ajouterdebut(L :^^noeud,N :^noeud)
   DÉBUT
   N⟶suiv ⟵ L^
   L^ ⟵ N
   FIN

4. **Procédure** ajoutfin(L :^^noeud,N :^noeud)
   VAR tmp : ^noeud
   DÉBUT
   SI (L^ = NIL) alors
   L^ ⟵ N
   N⟶suiv ⟵ NIL
   SINON
   tmp ⟵ L^
   Tant que (tmp⟶suiv ≠ NIL) faire
   tmp ⟵ tmp⟶suiv
   Fin tant que
   tmp⟶suiv ⟵ N
   N⟶suiv ⟵ NIL
   FinSI
   FIN

5. **Procédure** affichenoeuds(L :^noeud)
   VAR p :^noeud

DÉBUT

p ⟵ L

Tant que (p ≠ NIL) faire

Ecrire("@ :",p,"r: ",p⟶r,"@suiv :",p⟶suiv)

p ⟵ p⟶suiv

Fin tant que

FIN

6. **Fonction** compternoeuds(L :^noeud) : entier

VAR cn : entier

p :^noeud

DÉBUT

p ⟵ L

cn ⟵ 0

Tant que (p ≠ NIL) faire

cn ⟵ cn+1

p ⟵ p⟶suiv

Fin tantque

Renvoyer cn

FIN

7. **Procédure** supfin(L :^^noeud)

VAR tmp,sup :^noeud

DÉBUT

SI (L^ = NIL) alors

Écrire("impossible de supprimer")

SINON

tmp ⟵ L^

Tant que (tmp⟶suiv⟶suiv ≠ NIL) faire

tmp ⟵ tmp⟶suiv

Fin tant que

sup⟵ tmp⟶suiv

tmp⟶suiv ⟵ NIL

Libérer(sup)

FinSI

FIN

8. **Procédure** triernoeuds(L :^^noeud)

VAR i,j,min :^noeud

val :réel

DÉBUT

Pour i de L^ à suiv = NIL pas⟵i⟶suiv faire

min ⟵ i

Pour j de i⟶suiv à NIL pas⟵j⟶suiv faire
SI (j⟶r < min⟶r) alors
min ⟵ j
Fin SI
Fin Pour
SI (min ≠ i) alors
val ⟵ i⟶r
i⟶r ⟵ min⟶r
i⟶r ⟵ val
Fin SI
Fin Pour
FIN

Programmation C

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct noeud {
        float r;
        struct noeud *suiv;
}noeud;
noeud *creernoeud( ){
noeud *n=malloc(sizeof(noeud));
printf("Donner une valeur reelle");
scanf("%f",&n->r);
return n; }
void ajoutdebut(noeud **L,noeud *N){
N->suiv = *L;
*L = N;}
void ajoutfin(noeud **L,noeud *N){
noeud *tmp;
if (*L == NULL){
*L = N;
N->suiv = NULL;
}else{
tmp = *L;
while (tmp->suiv != NULL)
tmp =tmp->suiv;
tmp->suiv = N;
N->suiv = NULL;
}}
void affichenoeuds(noeud *L){
```

```
28 noeud *p = L;
29   while(p != NULL){
30   printf("%p -- %.0f --%p\n ",p,p->r,p->suiv);
31   p = p->suiv;} }
32 short compternoeuds(noeud *L){
33 short cn = 0;
34 noeud *p = L;
35   while(p != NULL){
36   cn++;
37   p = p->suiv;}
38 return cn; }
39 void supfin(noeud **L){
40 noeud *sup,*tmp;
41 if(*L == NULL) printf("Impossible de supprimer");
42 else{
43 tmp = *L;
44 while (tmp->suiv != NULL)
45 tmp =tmp->suiv;
46 sup = tmp->suiv;
47 tmp->suiv = NULL;
48 free(sup);
49 }}
50 void triernoeuds(noeud **L){
51 noeud *i,*j,*min;
52 float val;
53 for(i=*L;i->suiv!=NULL;i=i->suiv){
54     min = i;
55     for(j=i->suiv;j!=NULL;j=j->suiv)
56       if(j->r < min->r) min = j;
57     if (min != i){
58     val = min->r;
59     min->r = i->r;
60     i->r = val;
61     }}}
62 int main() {
63     noeud *Liste = NULL;
64     ajoutdebut(&Liste,creernoeud());
65     ajoutdebut(&Liste,creernoeud());
66     ajoutdebut(&Liste,creernoeud());
67     printf("\nNombre des noeuds %hi ",compternoeuds(Liste));
68     affichenoeuds(Liste);
```

```
69      supfin(&Liste);
70      ajoutfin(&Liste,creernoeud());
71      ajoutfin(&Liste,creernoeud());
72      printf("\nNombre des noeuds %hi ",compternoeuds(Liste));
73      triernoeuds(&Liste);
74      affichenoeuds(Liste);
75 return 0; }
```

Exercice N°2

Algorithmique

TYPE entierrs = **structure**
v : entier
suiv : ^**entierrs**
Fin structure

1. **Fonction** tabToList(T:TAB, n:entier) :^**entierrs**
   VAR L,N,tmp:^**entierrs**
   i:entier
   DÉBUT
   L ⟵NIL
   Pour i de 1 à n pas⟵ faire
   N ⟵allouer(**entierrs**)
   N⟶v ⟵T[i]
   N⟶suiv ⟵NIL
   SI (L = NIL) alors
   L ⟵ N
   SINON
   tmp ⟵ L
   Tant que (tmp⟶suiv ≠ NIL) faire
   tmp ⟵ tmp⟶suiv
   Fin tant que
   tmp⟶suiv ⟵ N
   FinSI
   Fin pour
   Renvoyer L
   FIN

2. **TYPE entierrs** = **structure**
   v : entier
   suiv : ^**entierrs**
   prec : ^**entierrs**

Fin structure
Fonction listToTab(L:^**entierrs**) :^entier
VAR T:^entier
p:^**entierrs**
i:entier
DÉBUT
T ⟵ NIL
p ⟵ L
i ⟵ 0
Tant que (p->suiv ≠ NIL) faire
p ⟵ p⟶suiv
Tant que (p->prec ≠ NIL) faire
SI (T = NIL) alors
T ⟵allouer(entier)
SINON
T ⟵reallouer(T,entier)
Fin SI
(T+i)^⟵ p⟶v
p ⟵ p⟶prec
i ⟵ i+1
Fin Tant que
Renvoyer T
FIN

Programmation C

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct entierrs {
        short v;
        struct entierrs *suiv;
}entierrs;
entierrs *tabToList(short T[ ],short n){
entierrs *L = NULL;
short i;
    for(i=0;i<n;i++){
        entierrs *N=malloc(sizeof(entierrs));
        N->v = T[i];
        N->suiv = L;
        L = N;}
 return L;}
 typedef struct entierrs {
```

```
17          short v;
18          struct entierrs *suiv;
19          struct entierrs *prec;
20 }entierrs;
21 short *listToTab(entierrs *L){
22 short i = 0;
23 short *T = NULL;
24 entierrs *p = L;
25  while(p->suiv != NULL)
26  p = p->suiv;
27   while(p->prec != NULL){
28     if (T == NULL) T = malloc(sizeof(entierrs));
29     else T = realloc(T,sizeof(entierrs));
30     *(T+i) = p->v;
31     i++;
32     p = p->prec;}
33 return T; }
34 void affichelist(entierrs *L){
35 while (L !=NULL){
36         printf("%p -- v: %hi ",L,L->v);
37         L = L->suiv;}
38 }
39 void afficheTab(short *T,short n){
40 short i = 0;
41 while (i < n){
42         printf("%hi ",*(T+i));
43         i++;}
44 }
45 int main() {
46     short Tab[]={23,5,20,7,11,9},*T = NULL;
47     entierrs *Liste = NULL;
48     Liste=tabToList(Tab,6);
49     affichelist(Liste);
50     T=listToTab(Liste);
51     afficheTab(T,6);
52 return 0; }
```

Exercice N°3

Algorithmique

1. TYPE **polynome** = **structure**
   cof : réel
   exp : entier
   suiv : ^**polynome**
   Fin structure

2. **Fonction** monome( ) :^polynome
   VAR M:^polynome
   DÉBUT
   M ⟵allouer(polynome)
   Écrire("donner un coeff ") Lire(M⟶cof)
   Écrire("donner un expo ") Lire(M⟶exp)
   M⟶suiv ⟵ NIL
   Renvoyer M
   FIN

3. **Procédure** ajoutenfin(P :^^polynome)
   VAR M,S :^polynome
   DÉBUT
   M ⟵ monome( )
   SI (P^ = NIL) alors
   P^ ⟵ M
   SINON
   S ⟵ P^
   Tant que (S⟶suiv ≠ NIL) faire
   S ⟵ S⟶suiv
   Fin Tant que
   S⟶suiv ⟵ M
   Fin SI
   FIN

4. **Procédure** affichepolynome(P :^polynome)
   VAR M :^polynome
   DÉBUT
   M ⟵ P
   Tant que (M ≠ NIL) faire
   SI (M⟶exp ≠ 0) alors
   Écrire(M⟶cof,"x^",M⟶exp,"+")
   SINON
   Écrire(M⟶cof)

Fin SI
M ⟵ M⟶suiv
Fin tant que
FIN

5. **Procédure** supprimedebut(P :^^polynome)
VAR S :^polynome
DÉBUT
S ⟵ P^
P^ ⟵ S⟶suiv
Libérer(S)
FIN

6. **Fonction** sommepoly(P1,P2 :^polynome) :^polynome
VAR P3,M:^polynome
DÉBUT
Tant que (P1 ≠ NIL OU P2 ≠ NIL) **faire**
M ⟵allouer(polynome)
SI (P1⟶exp = P2⟶exp) **alors**
M⟶exp ⟵ P1⟶exp
M⟶cof ⟵ P1⟶cof + P2⟶cof
P1 ⟵ P1⟶suiv
P2 ⟵ P2⟶suiv
SINON SI (P1⟶exp > P2⟶exp) **alors**
M⟶exp ⟵ P1⟶exp
M⟶cof ⟵ P1⟶cof
P1 ⟵ P1⟶suiv
SINON
M⟶exp ⟵ P2⟶exp
M⟶cof ⟵ P2⟶cof
P2 ⟵ P2⟶suiv
Fin SI
M⟶suiv ⟵ P3
P3 ⟵ M
Fin Tantque
Renvoyer P3
FIN

7. **Fonction** derivepoly(P :^polynome) :^polynome
VAR D,N,M :^polynome
DÉBUT
D ⟵ NIL
M ⟵ P
Tant que (M ≠ NIL) **faire**

SI (M⟶exp ≠ 0) **alors**
N ⟵allouer(polynome)
N⟶cof ⟵ M⟶exp * M⟶cof
N⟶exp ⟵ M⟶exp - 1
N⟶suiv ⟵ D
D ⟵ N
Fin SI
M ⟵ M⟶suiv
Fin Tantque
Renvoyer D
FIN

Programmation C

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct polynome{
        float cof;
        unsigned short exp;
        struct polynome *suiv;
}polynome;
polynome *monome( ){
polynome *M=malloc(sizeof(polynome));
printf("Donner coeff :");
scanf("%f",&M->cof);
printf("Donner expo :");
scanf("%hu",&M->exp);
M->suiv = NULL;
return M; }
void ajoutfin(polynome **P){
polynome *S,*M=monome();
    if (*P == NULL) *P = M;
    else{
    S = *P;
       while(S->suiv != NULL)
       S = S->suiv;
    S->suiv = M;}}
void affichepolynome(polynome *P){
polynome *M = P;
  while(M != NULL){
    if (M->exp != 0)
    printf("%.0fx^%hu +",M->cof,M->exp);
```

```
29      else
30      printf(" %.0f",M->cof);
31    M = M->suiv;} }
32 void supprimedebut(polynome **P){
33 polynome *S = *P;
34 *P = S->suiv;
35 free(S); }
36 polynome *sommepoly(polynome *P1,polynome *P2){
37 polynome *P3 = NULL,*M = NULL;
38   while (P1 != NULL || P2 != NULL){
39      M = malloc(sizeof(polynome));
40       if(P1->exp == P2->exp){
41          M->exp = P1->exp;
42          M->cof = P1->cof + P2->cof;
43          P1 = P1->suiv;
44          P2 = P2->suiv;}
45        else if(P1->exp > P2->exp){
46          M->exp = P1->exp;
47          M->cof = P1->cof;
48          P1 = P1->suiv;
49        }else{
50          M->exp = P2->exp;
51          M->cof = P2->cof;
52          P2 = P2->suiv;}
53        M->suiv = P3;
54        P3 = M;}
55 return P3;}
56 polynome *derivepoly(polynome *P){
57 polynome *D = NULL,*N,*M = P;
58   while (M != NULL){
59      if (M->exp != 0) {
60       N = malloc(sizeof(polynome));
61       N->cof = M->exp * M->cof;
62       N->exp = M->exp -1;
63       N->suiv = D;
64       D = N;}
65     M = M->suiv;}
66 return D;}
67 int main() {
68     polynome *Liste1 = NULL,*Liste2 = NULL;
69     ajoutfin(&Liste1);
```

```
70      ajoutfin(&Liste1);
71      ajoutfin(&Liste1);
72      ajoutfin(&Liste1);
73      ajoutfin(&Liste2);
74      ajoutfin(&Liste2);
75      ajoutfin(&Liste2);
76      affichepolynome(Liste1);
77      printf("\nSomme : ");
78      affichepolynome(sommepoly(Liste1,Liste2));
79      supprimedebut(&Liste1);
80      affichepolynome(Liste1);
81      printf("\nDerive : ");
82      affichepolynome(derivepoly(Liste2));
83  return 0; }
```